

```
$ cat notes.md
```

DevOps & Cloud [SWE] — Class 12 Notes

LECTURE TITLE	Deployment Strategies & Rollouts (<i>see the note below — the folder name doesn't match what was taught</i>)
DATE/TIME	10 September 2026, 2:00 PM
DURATION	120 minutes
INSTRUCTOR	Nensi Ravaliya
BIG IDEA	A Deployment isn't just "a ReplicaSet with extra steps" — it's the only object that remembers your history, which is the only reason you can undo a bad release.

WHAT'S INSIDE — 11 SECTIONS

- 00 ⚡ The 60-Second Version
- 01 📅 Housekeeping
- 02 👤 DaemonSet — Finally Explained
- 03 💎 Why Deployment Exists — The Question That Explains Everything
- 04 🔄 Rolling Update — The Two Numbers, Frame by Frame
- 05 🏗️ Lab — The Full Rollout Cycle
- 06 🏷️ Labels & Selectors — Where Each One Goes
- 07 📜 The Four Deployment Strategies
- 08 📄 Command Cheat-Sheet
- 09 📝 Homework — Five Tasks
- 10 📄 Notes on the Transcript Itself

Rebuilt from the raw class transcript — nothing cut, just organised, de-noised and made readable. `theme:` `Ember`

⚠ WATCH OUT

Read this first — two things about this class.

- 1. The folder name is wrong.** It says *"Ingress, ConfigMaps & Secrets"*, and so does the transcript header. **Those topics were not taught.** She said early on: *"If you are able to do this thing, then we will move forward with session 11 — ConfigMap, Secret and service types. I will try to cover."* She never got there. **The entire class was rolling updates and deployment strategies.** ConfigMaps, Secrets and Ingress are still ahead of you.
- 2. Roughly half the recording is unusable.** The second hour was hands-on lab time — she walked the room desk to desk while students ran commands — and the transcriber produced mostly noise, Portuguese and Spanish filler. Everything below comes from the ~50 usable minutes; where a stretch was too degraded to reconstruct, this says so rather than guessing.



00 ⚡ The 60-Second Version

- 1. ReplicaSet can scale. Only a Deployment can *undo*.** That's the whole reason Deployments exist.
- 2. `maxSurge`** = how many **extra** pods you may create during an update. **`maxUnavailable`** = how many you may drop below target.
- 3. A rolling update is a **shuffle, not a switch**:** add one new pod → remove one old pod → repeat, so the app never goes down.
- 4. Every `kubectl apply` creates a revision.** Revision 1, 2, 3... That's your undo history.
- 5. Three commands run your releases:** `rollout status` · `rollout history` · `rollout undo` .
- 6. Four deployment strategies:** RollingUpdate (default) · Blue-Green · Canary · Recreate.
- 7. Labels + selectors are how everything finds everything.** `kubectl get pods -l app=app-rolling --show-labels` .
- 8. A DaemonSet puts one pod on every node** — and automatically on any new node that joins.



01 📅 Housekeeping

😊 The homework reckoning

She opened by asking who'd finished the v2 deployment homework. The silence was audible.

CLASS MOMENT

"When I asked you — who did, who did not — nobody said anything. Just share the screen."

Then, resigned: *"Okay guys, I will try to cover all this thing. Depends on how much response I get from you."*

NOTE

Note on repetition: she re-teaches **maxSurge / maxUnavailable**, the **deployment strategies** and **ReplicaSet vs Deployment** here, having covered them in Class 11. **These notes cover them in full again rather than pointing back** — because she does, and because the second pass uses a different worked example. If it clicked the first time, skim §3–§4; if it didn't, this version may be the one that lands.

| The Kubernetes story so far

Her own recap of everything covered:



```
Session 1 → core components: master + worker node, how they work together
           basic commands: get, describe, logs, cluster-info
Session 2 → pod.yaml, ReplicaSet, why Deployment, StatefulSet, DaemonSet
           scaling replicas up and down
Session 3 → (today) rolling updates + rollout commands
```

| She changed how the class runs

This is worth knowing because it affects every session from here:

SAID IN CLASS

"I will create a readme file and all the resources BEFORE the session, and I'll tell you — so you can go through the readme and all the concepts. Then the next day we directly do hands-on."

"So you'll learn fast, and if you have a doubt you can tell me directly: 'this command I tried last time and it didn't work.' We can resolve it. Is it good?"

The class agreed. 😊 *"I just tried a new approach — I will run the command and you will see... today I can see by the faces that you completed it. At least I can see in your eyes. This is how a teacher can read the eyes and get to know: this person is doing, or this person is ignoring."*

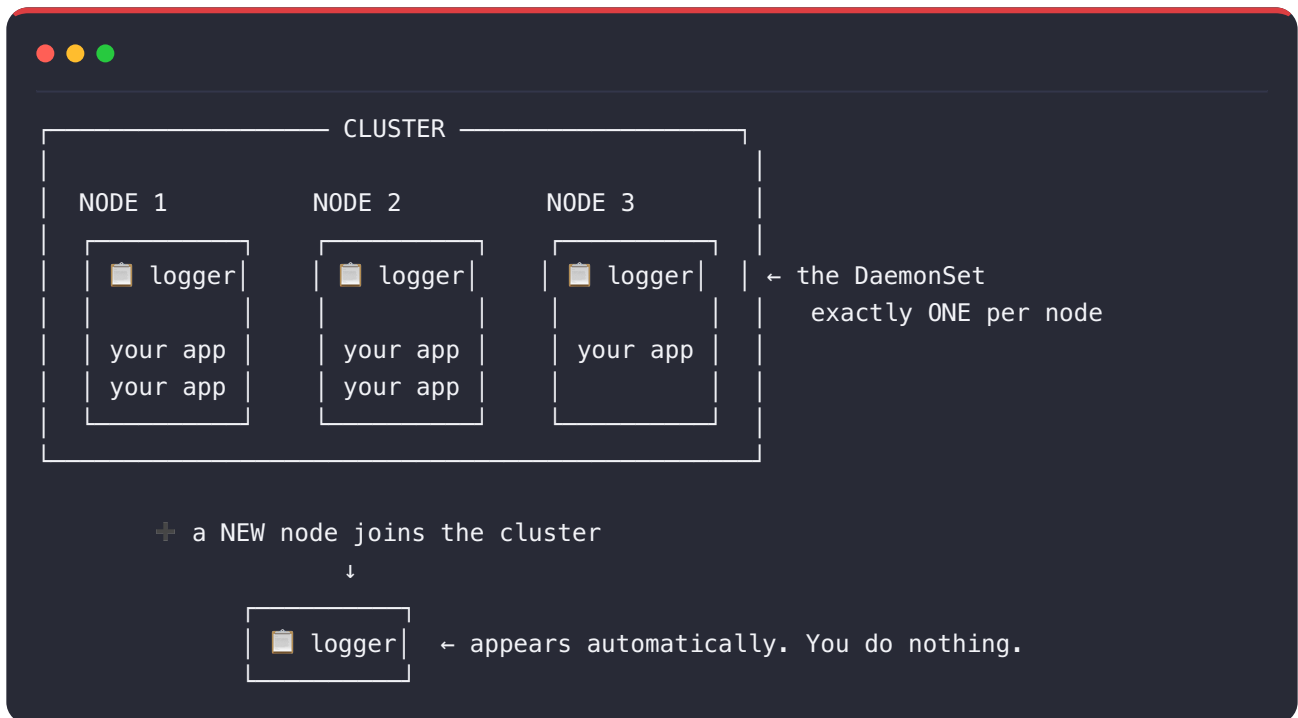


02 DaemonSet — Finally Explained

Section B apparently never got this, so she covered it properly.

SAID IN CLASS

"What do you mean by DaemonSet? Tell me." Student: "A pod that we run in each and every node." ✓



LOCK THIS IN

Her example: "Let's suppose I want to create a log collector. It's basically a simple container that I want to run inside each node. Let's say I have 10 servers... I need to deploy that one container in all 10."

🎯 **And the key property:** "After running this in each node — let's suppose a new server is added — then it will automatically create that pod inside the new server. That's what a DaemonSet is."

PICTURE THIS

The one-line definition she landed on: "Nothing else — a DaemonSet is just a container who is running everywhere inside your cluster."

GOOD TO KNOW

Typical uses: log collectors (Fluentd, Filebeat) · security agents (**Falco**, from Class 10) · monitoring (`node-exporter`) · networking plugins.

⚠️ **Why she couldn't demo it:** "DaemonSet — once you get a server. At least two servers we need for hands-on. Otherwise how will you know it creates one in each? You can run the DaemonSet file and it will work, but there's nothing you can visibly see." On a one-node minikube, a DaemonSet just makes one pod — which looks identical to a plain pod.



03



Why Deployment Exists — The Question That Explains Everything

INTERVIEW ALERT

She asked it directly: *"Why do we need a Deployment, if we have a ReplicaSet?"*

Student: *"Scaling." → "Scaling we have in ReplicaSet. But why do we need Deployment?"*

Student: *"To manage migrations." →  "Perfect. That's what we are doing."*

The thing ReplicaSet simply cannot do

SAID IN CLASS

"Are you able to see the history with a ReplicaSet? No — you can only scale down and scale up. Do you know a history command in ReplicaSet? There's no command. Either I can scale up, either I can scale down."

WITH ONLY A REPLICASET

1. edit the file manually
2. scale ALL old replicas down
3. create the new ones
4. hope it works

↓

new version is broken?

↓

✗ NO WAY BACK.

rebuild it by hand.

WITH A DEPLOYMENT

1. edit the image tag
2. `kubectl apply`

↓

done. no downtime.

new version is broken?

↓

✓ `kubectl rollout undo`
...back in 3 seconds.

SAID IN CLASS

"What if my new deployment is not working and I want to roll back to the old one? What will I do? There is no way in ReplicaSet that you can roll back from new to old. That's why we need Deployment. That's why."

✓ LOCK THIS IN

Lock this in — the interview answer:

	ReplicaSet	Deployment
Keep N pods alive	✓	✓
Scale up / down	✓	✓
Rolling update	✗	✓
Revision history	✗	✓
Rollback	✗	✓

A Deployment manages ReplicaSets. You write the Deployment; it creates a ReplicaSet per version and shifts pods between them.



04 🎬 Rolling Update — The Two Numbers, Frame by Frame

This was the centrepiece, and she walked the whole animation on the board.

```
spec:
  replicas: 4
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # how many EXTRA pods I may create
      maxUnavailable: 0 # how many I may go BELOW target
```

The two numbers, in her words

Field	Her definition	With replicas: 4
maxSurge	"Maximum number of replicas that we can have during the migration from old to new."	$4 + 1 = 5$ max
maxUnavailable	"Bare minimum replicas that need to be up and running when you're migrating from old to new."	$4 - 0 = 4$ min

PICTURE THIS

Read them as a ceiling and a floor:



```
▲ 5 pods ————— maxSurge ceiling (4 + 1)
|
| 4 pods == target ==
|
▼ 4 pods ————— maxUnavailable floor (4 - 0)
```

The update is only ever allowed to move INSIDE this band.



The walkthrough she drew — 3 replicas, maxSurge 1, maxUnavailable 1



WATCH IT HAPPEN

Migrating v1 → v2. Watch the count.



START		3 old. Target 3, ceiling 4.
STEP 1 +new		4 pods – at the ceiling. "I created one with the new version."
STEP 2 –old		3 pods – back to target. ⚠️ MUST come down before going up again.
STEP 3 +new		4 pods – ceiling again.
STEP 4 –old		3 pods.
STEP 5 +new		4 pods.
STEP 6 –old		3 pods. ✅ DONE. All new.
	= v1 (old) = v2 (new)	

INTERVIEW ALERT

The bit she made the class work out. After step 1 you have 4 pods. Do you add another, or remove one?

Half the room said *"plus one."* Her correction:

"Why do we need to do minus? Because right now I have 4 replicas. If I do plus 1 that means 5 replicas. But maxSurge says maximum to maximum we can have only four. I can't have five. That's why I need to bring down one replica, and then create a new one. So it balances at four."

✅ **The rule: you can never exceed the ceiling, and never drop below the floor.** Every step alternates to stay inside the band.

CLASS MOMENT

When the room stalled: *"Other people are confused, asleep, or not doing it."* 😊

WHY IT WORKS

What the two numbers actually control — worth knowing before you pick values:

Setting	Behaviour	Cost
<code>maxSurge: 0,</code> <code>maxUnavailable: 1</code>	remove first, then add	cheapest — no extra capacity, but you run degraded
<code>maxSurge: 1,</code> <code>maxUnavailable: 0</code>	add first, then remove	★ safest — full capacity throughout, needs room for 1 extra
<code>maxSurge: 100%,</code> <code>maxUnavailable: 0</code>	all new pods at once, then drop all old	fastest — needs double the resources

✅ **maxSurge: 1, maxUnavailable: 0 is the sane default for production**, and it's exactly what her `deployment-v1.yaml` uses. Never fewer pods than you promised; at most one extra machine's worth of cost.



05 🏆 Lab — The Full Rollout Cycle

This is the practical core of the class. Do all of it.



DO THIS YOURSELF

Lab 1 — deploy v1 and expose it.



```
minikube start

kubectl apply -f deployment-v1.yaml
kubectl apply -f service.yaml

kubectl get all          # ★ deployment + replicaset + pods + service, all at once
```



kubectl get all is the command to remember — she used it constantly. One command, the whole picture.



And notice what appears that you never created: a ReplicaSet. *"From the deployment we mentioned replicas, so it will automatically create my ReplicaSet."* That's §3's layering, visible.



DO THIS YOURSELF

Lab 2 — check the rollout status.



```
kubectl rollout status deploy/app-rolling
# → deployment "app-rolling" successfully rolled out
```



DO THIS YOURSELF

Lab 3 — look at the labels.



```
kubectl get pods -l app=app-rolling --show-labels
```



```
-l app=app-rolling    ← FILTER: only pods with this label
  |   |
  |   └── value
  └── key

--show-labels        ← also print every label each pod carries
```

You'll see all 4 pods carrying **version=v1**.



Why this matters at scale: *"If you want to search 50 particular pods out of 10,000 pods by label, you can use this."*



DO THIS YOURSELF

Lab 4 — ship v2.

Copy `deployment-v1.yaml` → `deployment-v2.yaml` and change **exactly two things**:

```
labels:
  version: "2.0.0"      # was 1.0.0
...
image: nginx:1.25      # was nginx:1.24
```

```
kubectl apply -f deployment-v2.yaml
kubectl get all          # watch: ContainerCreating → Running
kubectl get pods -l app=app-rolling --show-labels    # → version=v2 ✓
```

 "This is my change in my new version. Now I'm creating a second file with version v2, and image 1.25. Previously it was 1.24."



DO THIS YOURSELF

Lab 5 — the history.

```
kubectl rollout history deploy/app-rolling
# REVISION  CHANGE-CAUSE
# 1          <none>
# 2          <none>
```

 **What a "revision" is, in her words:** "Revision means how many times you ran this apply command. I ran it one time — I see revision 1. If I apply the same file again, this will change from 1 to 2."

 **Careful with that** — see the clarification below.



DO THIS YOURSELF

Lab 6 — the undo. This is the payoff.

```
kubectl rollout undo deploy/app-rolling
# → deployment.apps/app-rolling rolled back

kubectl get pods -l app=app-rolling --show-labels
# → version=v1 ✓ you are back on the old version
```



"That means from the v2 tag we rolled back to v1. We rolled out to the old deployment."

😅 She fumbled the typing live: "What happened? Typo. You can expect a typo — that's why I know that I'm wrong." 😊



CLARIFIED

One clarification on revisions, because her phrasing will mislead you.

A revision is created when the **pod template actually changes** — not on every **apply**. Re-applying an identical file produces *no* new revision, because nothing changed. Change the image tag or a label → new revision.

Why it matters: if you **apply** the same file five times expecting revisions 1–5 and then **rollout undo**, you'll jump back further than you expected. Check with **rollout history** before undoing, never assume.



And a tip she didn't mention: that empty **CHANGE-CAUSE** column can be filled in, which makes history readable months later:

```
kubectl annotate deploy/app-rolling kubernetes.io/change-cause="upgrade nginx 1.24 → 1.25"
```

🏆 Lab 7 — her extension task

🏆 DO THIS YOURSELF

"Create two more files — `deployment-v3` and `deployment-v4` — changing the `nginx` image tag. Then roll back from `v4` to `v1`, or `v4` to `v2`."

⚠️ **The catch she set deliberately:** "The `undo` command that we did — you need to change that command. Search for it."

✅ **The answer she wanted you to find:**

```
kubectl rollout history deploy/app-rolling      # find the revision number
kubectl rollout undo deploy/app-rolling --to-revision=1  # ← the extra flag
```

Plain `undo` only steps back **one** revision. `--to-revision=N` jumps to any point in the history.



06 📁 Labels & Selectors — Where Each One Goes

She quizzed the class on this and it's worth pinning down, because the two words get swapped constantly.

	Goes in	Means
labels	<code>metadata:</code> of a Pod (or a template)	"here is a sticker describing me"
selector	<code>spec:</code> of a Service, ReplicaSet, Deployment	"I manage anything wearing that sticker"

```
POD                                SERVICE / REPLICASET / DEPLOYMENT
metadata:                         spec:
  labels:                         selector:
    app: nginx                    app: nginx
    └─ the sticker                └─ the search query
```

🗣️ SAID IN CLASS

"Where are we adding the label — `pod.yml` or `service.yml` ? `pod.yml`. And selector? Selector in `service`, and in `ReplicaSet` and `Deployment`. Everywhere we are using `selector`." ✅

🧩 WHY IT WORKS

Why it's built this way: nothing in Kubernetes holds a *list* of the things it owns. Everything is a **live query**. That's why a Service keeps working when a Deployment destroys and recreates every pod underneath it — the new pods wear the same sticker, so the query still matches. No IPs, no lists, no updating anything.



07 🎬 The Four Deployment Strategies

🎤 SAID IN CLASS

"There are four types of deployment. One is rolling update, that we're doing right now. Second is blue-green. Third, canary. Fourth, recreate."

🔥 **And her position on the syllabus:** "We're going to cover the first one. Second, third and fourth are out of syllabus. But if you're planning for a DevOps engineer job role, then this is **NOT** out of syllabus. This is meant to be. That's why I added it."

😄 "Whether it's in your syllabus or not, I don't care. You have to do it."

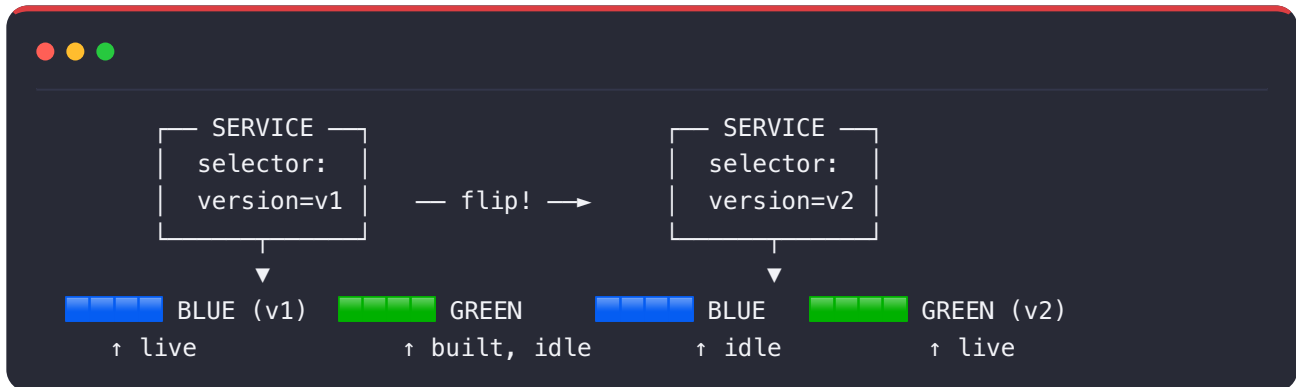
📌 She demonstrated RollingUpdate live and set the rest as homework, so the diagrams below expand her assignment — the strategies are hers, the illustrations are here to make them learnable before you attempt them.

1 RollingUpdate — replace them gradually ⭐ the default



Downtime	none
Extra cost	one extra pod (with <code>maxSurge: 1</code>)
Watch out	v1 and v2 serve traffic simultaneously — your API must tolerate that
Use when	almost always. It's the default for a reason.

2 Blue-Green — build the new one beside it, then flip



Downtime none — the switch is instant

Extra cost ⚠️ **double** — both versions fully running

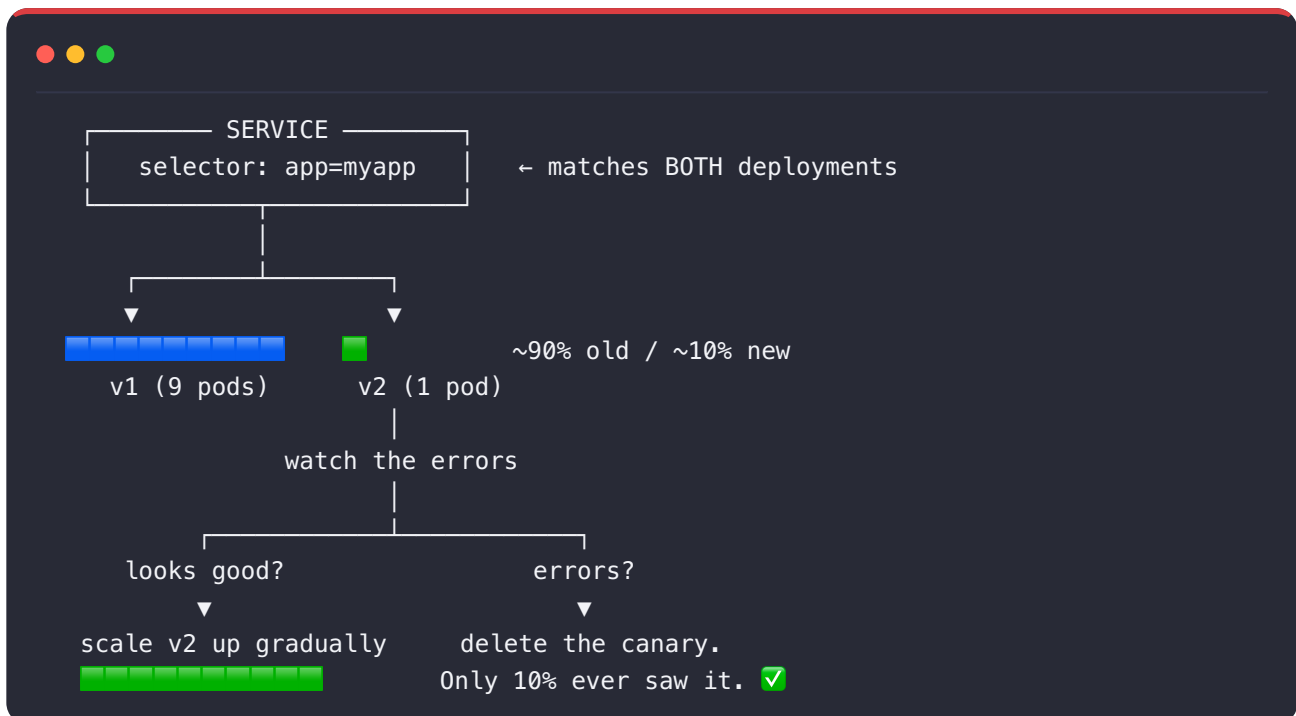
Rollback ⭐ **instant** — flip the selector back

Use when you cannot tolerate mixed versions, and rollback speed matters most

✓ LOCK THIS IN

The mechanic is just a label edit. The Service's `selector` points at `version: v1`; you change it to `v2`. Every user moves in one instant, and blue is still sitting there if you need it back.

3 Canary — let a few users test it for you 🤖

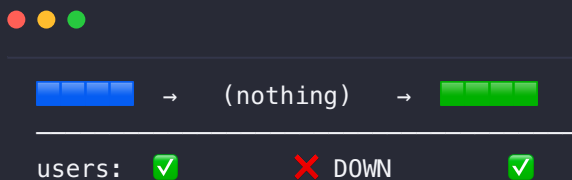


PICTURE THIS

Why "canary": miners took a canary down the shaft. If the bird stopped singing, the air was bad — and only the bird paid for finding out.

Downtime	none
Extra cost	small — one extra pod
Superpower	limits the blast radius. A bad release hits 10% of users, not 100%
Use when	the change is risky and you want real traffic to prove it

4 Recreate — kill everything, then start 💀



Downtime	⚠️ yes — real, visible downtime
Extra cost	none
Use when	v1 and v2 genuinely cannot coexist — e.g. a database migration that changes the schema, or an app that takes an exclusive file lock

```
spec:
  strategy:
    type: Recreate    # no rollingUpdate block – there's nothing to tune
```

Pick one

Strategy	Downtime	Cost	Rollback	Mixed versions?
RollingUpdate	none	+1 pod	gradual	⚠️ yes
Blue-Green	none	2×	★ instant	❌ no
Canary	none	+1 pod	delete the canary	⚠️ yes, on purpose
Recreate	⚠️ yes	none	redeploy old	❌ no





Rollouts — the three that matter



```
kubectl rollout status deploy/<name>      # is it finished?
kubectl rollout history deploy/<name>      # what revisions exist?
kubectl rollout undo deploy/<name>         # back one revision
kubectl rollout undo deploy/<name> --to-revision=2 # back to a specific one
kubectl rollout restart deploy/<name>      # recycle all pods, same version
```



SAID IN CLASS

"Make sure you remember the commands — rollout status, rollout undo, rollout history. These three commands you need hands-on. That's enough for you."

Seeing what's there



```
kubectl get all                                # ★ everything in this namespace
kubectl get pods -l app=app-rolling           # filter by label
kubectl get pods -l app=app-rolling --show-labels # filter + show all labels
kubectl get pods -o wide                      # + IP and node
kubectl describe deploy/<name>                 # full detail incl. strategy
```

Applying & cleaning up



```
kubectl apply -f deployment-v1.yaml
kubectl apply -f service.yaml
kubectl delete all --all # ⚠️ wipes everything in the namespace
```



WATCH OUT

kubectl delete all --all does exactly what it says — every pod, deployment, replicaset and service in the current namespace. She used it to reset between demos. Great in minikube, catastrophic anywhere real.

The deployment YAML that drives it all

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: yatri-backend
  labels:
    app: yatri-backend
    version: "1.0.0"          # ← bump this per release
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1             # ← the ceiling
      maxUnavailable: 0      # ← the floor
  selector:
    matchLabels:
      app: yatri-backend      # ← must match template labels
  template:
    metadata:
      labels:
        app: yatri-backend
        version: "1.0.0"
    spec:
      containers:
        - name: backend
          image: python:3.11-alpine3.19  # ← and bump this
          ports:
            - containerPort: 5000
          resources:
            requests:          # what it's guaranteed
              cpu: "100m"
              memory: "128Mi"
            limits:            # what it can never exceed
              cpu: "250m"
              memory: "256Mi"
```

WHY IT WORKS

requests vs limits — this is the field the scheduler reads.

- **requests** = "reserve this much for me." This is what the **scheduler** uses to pick a node (Class 10 §4 — "node 1 only has 5 GB free, no").
- **limits** = "never let me exceed this." Go over the memory limit and the container is **killed** (**OOMKilled**); go over CPU and you're just **throttled**.
- **100m** = 100 **millicores** = **0.1 of one CPU core**. **1000m** = 1 full core.

⚠️ **Pods with no requests** are the classic production incident: the scheduler assumes they need nothing, packs a node full, and everything on it starts starving.



09



Homework — Five Tasks

She struggled to remember the whole list 😊 — "I had five tasks. I remember three. If someone asks me — I forget, don't ping me on WhatsApp that I forgot my homework task." Here's the reconstructed set:

#	Task
1	📖 Difference between StatefulSet, Deployment and DaemonSet
2	📖 Difference between ReplicaSet and Deployment (<i>answer in §3</i>)
3	🏆 The other deployment strategies — Blue-Green (~50% done in class), then Canary and Recreate , hands-on
4	🔗 The five Service types — she's pre-built the files: ClusterIP · NodePort · LoadBalancer · ExternalName · Headless . "In each service I've given the <i>deployment.yml</i> , the <i>pod.yml</i> , the <i>service</i> , and the <i>readme</i> — how to deploy it, why you need it, in which scenarios, and the main points they ask in interviews."
5	🌐 Research FQDN and CoreDNS and write a readme explaining both

NOTE

Why task 5 matters: "Next session we have a DNS setup and test cases — for that you need FQDN understanding."

💡 **FQDN** = Fully Qualified Domain Name. In Kubernetes, every Service gets one, and it follows a fixed pattern:

```
my-service . my-namespace . svc . cluster.local
  |         |         |         |
  |         |         |         |
service   namespace  type    cluster suffix
```

CoreDNS is the component inside the cluster that resolves those names. It's why `http://backend:5000` worked in Class 9 without a single IP address.

INTERVIEW ALERT

She also mentioned a bigger piece coming: "I was going to cover today's project — I was going to give a big demo project. No matter what, next time."



10 📅 Notes on the Transcript Itself

- **Instructor name:** the header credits "Ritesh Prajapati" — the **Scaler++ Chrome extension's developer**, not the teacher. The instructor is **Nensi Ravaliya** ([@Nency-Ravaliya](#)).
- ⚠️ **The lecture title is wrong.** Both the folder name and the transcript header say "Ingress, ConfigMaps & Secrets". Those were the *planned* topics — she says so — but the class never reached them. **What was actually taught: DaemonSets, rolling updates, rollout commands and deployment strategies.** Ingress, ConfigMaps and Secrets are still outstanding.
- ⚠️ **About half this recording is unusable.** The first ~50 minutes are coherent; after she sets the lab and starts walking the room, the transcript collapses into repeated "Thank you", Portuguese and Spanish filler, and fragmentary half-sentences. Anything that couldn't be reconstructed with confidence has been left out rather than guessed at.
- 🔍 **One clarification added:** her definition of a **revision** ("how many times you ran apply") is close but not exact — revisions are created when the **pod template changes**, which matters if you rely on `rollout undo`. Her version is first, the correction is in §5.
- 📌 **Session numbering differs from folder numbering.** She refers to this material as "session 10 / session 11"; the folders here are `Class-12`. Her `session11` content — the five Service types — is your homework task 4.
- 📌 **Her own examples used two naming schemes:** the live demo deployment is `app-rolling`, while the YAML files in the repo use `yatri-backend`. Same structure, different names.

- Garbled terms corrected: "Qtl / cubectl / CTL / cube CDL" → **kubectl** , "max search" → **maxSurge** , "demon set / demo set" → **DaemonSet** , "MiniCube" → **minikube** , "rollout undo / unto command" → **rollout undo** , "Blue River" → **blue-green** , "deployment-b3 / b2" → **deployment-v3 / v2** , "FQDM" → **FQDN** , "pod.tml / service.tml" → **pod.yml / service.yml** .